Master in Data Science – Semantic Data Management 25/26

Knowledge Graphs: Laboratory 2 Report

Davide Volpi
davide.volpi@estudiantat.upc.edu

Federico Clerici
federico.clerici@estudiantat.upc.edu

A. Knowledge Graph Modeling

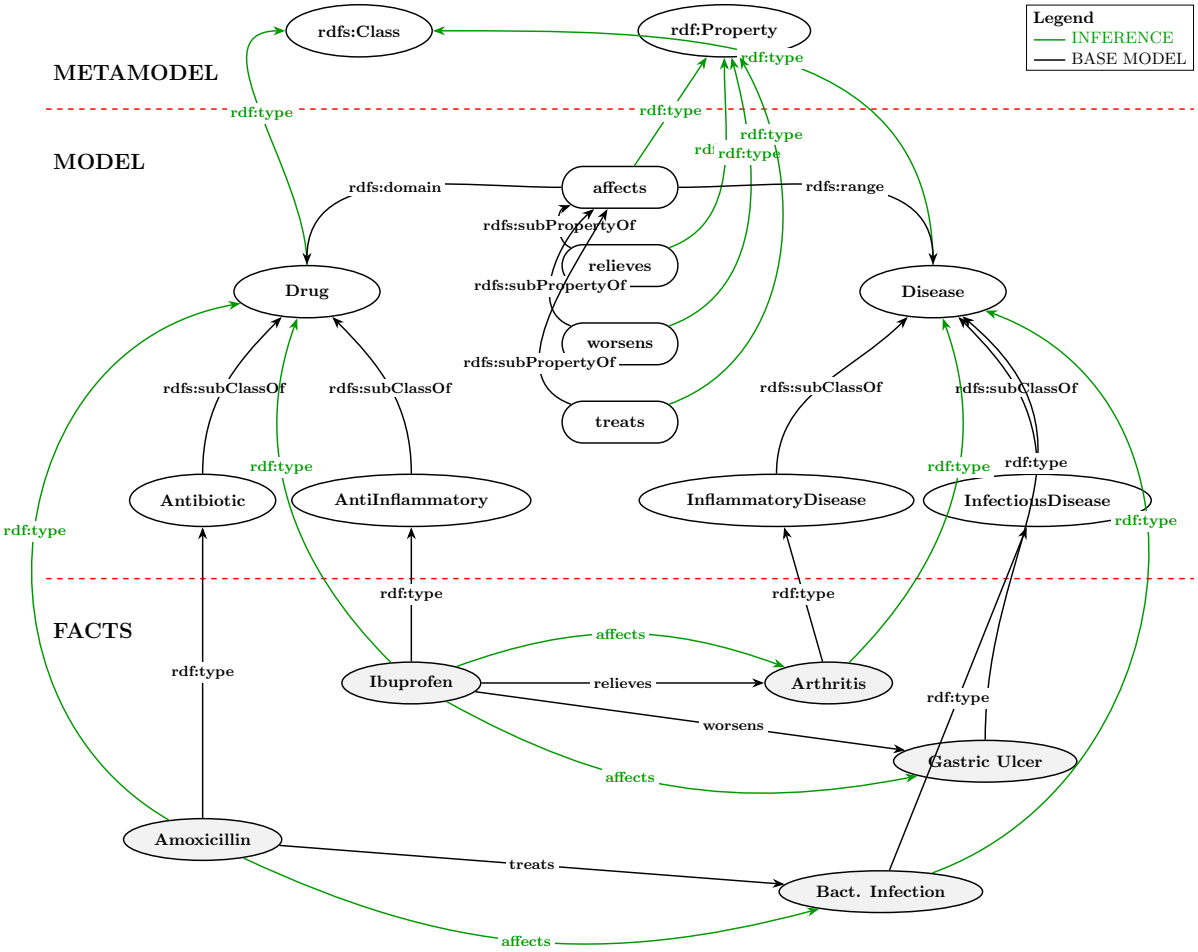

Figure 1: Knowledge Graph modelling using RDFS logic.

Justification of Modeling Decisions

To capture the biomedical knowledge domain accurately and promote data interoperability, the RDFS Knowledge Graph was structured with the following key modeling decisions:

- **Layer Separation (Metamodel, Model, and Facts):** The graph explicitly distinguishes between the standard Metamodel (`rdfs:Class`, `rdf:Property`), the conceptual biomedical schema (Model), and the concrete drug/disease facts (Instances). This adheres to semantic web best practices, ensuring the dataset is interpretable and extensible.
- **Hierarchical Class Abstraction:** Entity types were modeled using `rdfs:subClassOf`. For example, `Antibiotic` and `AntiInflammatory` are specific subclasses of `Drug`. This structural

rule guarantees that any instance classified as an `Antibiotic` is automatically inferred by the reasoner to also be a `Drug`.

- **Property Hierarchies for Generalized Knowledge:** To fulfil the requirement of capturing the general notion of a drug "affecting" a disease, a high-level property `affects` was introduced. The specific interaction properties (`treats`, `relieves`, `worsens`) were modelled as sub-properties (`rdfs:subPropertyOf`) of the `affects` parent property.
- **Zero-Redundancy via Domain/Range Inheritance:** To minimize redundancy, the `rdfs:domain` (`Drug`) and `rdfs:range` (`Disease`) constraints were defined only on the parent `affects` property. Because `treats`, `relieves`, and `worsens` are declared as sub-properties, they automatically inherit these domain and range constraints through RDFS inference.
- **Instance Mapping:** Factual instances were explicitly mapped to their most specific classes using `rdf:type`. Specific relationships (e.g., Amoxicillin treats Bacterial Infection) were asserted, leaving the broader inferred abstractions (like the `affects` relationship or the `Drug` superclass assignments) to be materialized by GraphDB's inference engine.

Modeling Aspects Missing from the Neo4j Solution

The provided Neo4j property graph is designed to assert isolated, rigid facts and falls short when representing generalized conceptual knowledge. The proposed RDFS model rectifies several deficits inherent to the Neo4j approach:

- **Lack of Property Hierarchies:** The Neo4j query asserts discrete, flat relationships (`[:TREATS]`, `[:RELIEVES]`, `[:WORSENS]`). It possesses no native semantic mechanism to declare that these relationships are specific derivations of a broader `AFFECTS` interaction. The RDFS model seamlessly handles this via `rdfs:subPropertyOf`.
- **Lack of Class Hierarchies:** The Neo4j solution assigns multiple labels to a node simultaneously (e.g., `(ibuprofen:Drug:AntiInflammatory)`). While this tags the data instance, it does not create a universal logical schema rule stating that all Anti-inflammatories are inherently Drugs. The RDFS model achieves this universal logic using `rdfs:subClassOf`.
- **Absence of Automated Inference:** In Neo4j, a user looking for "drugs that affect diseases" would have to explicitly hardcode every specific relationship type into their query (e.g., `MATCH (d)-[:TREATS|RELIEVES|WORSENS]->(ds)`). In an RDFS Knowledge Graph, the system reasons over the data; querying for the `affects` property will automatically return all inferred sub-interactions without needing to manually list them.

B. Knowledge Graphs in GraphDB

Graph Generation and Instantiation Methodology

To create the designed RDFS schema and populate the Knowledge Graph, we used the Python `rdflib` library. The implementation focused on the following key aspects:

- **Programmatic Schema Definition:** The Metamodel and Model layers were strictly defined in Python.

- **Dataset Expansion:** To guarantee the graph was rich for expressive SPARQL querying, the base schema was extended with additional subclasses (e.g., `Antiviral` and `ViralInfection`). A deterministic randomized pipeline was then implemented to generate 50 distinct dummy drugs and 50 dummy diseases, yielding approximately 100 interconnected semantic factual relationships (`treats`, `relieves`, `worsens`).
- **Zero-Redundancy via Delegated Reasoning:** In adherence to the assignment's core quality criterion, the generation script was optimized to assert only explicit base facts. Redundant triples, such as asserting that an `Antibiotic` instance is also a `Drug` or explicitly writing the parent `affects` relationships, were intentionally excluded from the code. The materialization of these inferences is delegated to GraphDB's RDFS reasoning engine.
- **Serialization:** The finalized graph was serialized into the Turtle (`.ttl`) syntax, providing format for import into the GraphDB repository.

C. Querying the Knowledge Graph in GraphDB

The following SPARQL queries are executed in GraphDB with RDFS/OWL reasoning enabled. We make the following assumptions:

- Reasoning is enabled in GraphDB, so that instances of subclasses of `ex:Drug` and `ex:Disease` are included in results via `rdfs:subClassOf` inference, not just directly typed instances.
- `ex:affects` is a defined object property in the ontology relating `ex:Drug` to `ex:Disease`. The relationship is directed: a drug affects a disease, not the other way around.
- All drugs and diseases in the Knowledge Graph use the `ex:` namespace. Resources from external ontologies would not be returned unless explicitly mapped.
- If an inverse property (e.g., `ex:isAffectedBy`) is declared in the ontology, reasoning will automatically infer the inverse triples as well.

C.1 List All Drugs

SPARQL:

```
PREFIX ex: <http://example.org/biomed#>

SELECT ?drug
WHERE {
  ?drug a ex:Drug .
}
```

Neo4j Cypher:

```
MATCH (d:Drug)
RETURN d
```

Both queries filter nodes by type: SPARQL does so via the `rdf:type` triple pattern (`?drug a ex:Drug`), while Cypher uses a node label constraint (`:Drug`) directly in the `MATCH` pattern. The key difference is that in RDF, type is just another triple stored in the graph, whereas in Neo4j, labels are a native indexing mechanism stored separately from relationships.

C.2 List All Diseases

SPARQL:

```
PREFIX ex: <http://example.org/biomed#>

SELECT ?disease
WHERE {
  ?disease a ex:Disease .
}
```

Neo4j Cypher:

```
MATCH (d:Disease)
RETURN d
```

Both queries filter nodes by type, mirroring the same mechanism as Query 1: SPARQL matches via the `rdf:type` triple pattern (`?disease a ex:Disease`), while Cypher uses the `:Disease` label constraint in the `MATCH` clause. Again, in RDF, being a `Disease` is an explicit fact stored as a triple in the graph, whereas in Neo4j it is a structural label assigned at node creation.

C.3 Retrieve all pairs of drugs and diseases such that the drug affects the disease

SPARQL:

```
PREFIX ex: <http://example.org/biomed#>

SELECT ?drug ?disease
WHERE {
  ?disease a ex:Disease .
  ?drug    a ex:Drug .
  ?drug    ex:affects ?disease .
}
```

Neo4j Cypher:

```
MATCH (drug:Drug)-[:RELIEVES|TREATS|WORSENS]->(disease:Disease)
RETURN drug.name, disease.name
```

Both queries retrieve pairs of nodes where a drug is related to a disease. In SPARQL, the relationship is captured via a single semantic property `ex:affects`, which is defined in the ontology and connects `ex:Drug` instances to `ex:Disease` instances. The directionality is encoded in the triple pattern itself: the drug appears as the subject and the disease as the object. In Neo4j Cypher, there is no single `affects` relationship type. Instead, the data models the specific nature of each interaction using three distinct relationship types: `RELIEVES`, `TREATS`, and `WORSENS`. The `|` operator allows matching any of these types in a single pattern. Directionality is enforced structurally by the arrow `-->` in the pattern, ensuring the drug is always on the left (source) and the disease on the right (target). A further difference arises from the use of reasoning in GraphDB: since `ex:affects` is a defined object property in the ontology, any drug subclass instance (e.g., an `ex:Antibiotic`) will also be returned via `rdfs:subClassOf` inference, without needing to explicitly list subtypes. In Neo4j, no such inference mechanism exists, all relevant relationship types must be enumerated explicitly in the query.

C.4 Inferences and Triple Patterns Generated by GraphDB

For each of the queries executed above, GraphDB relies on its RDFS (Optimized) reasoning engine to materialize implicit triples. This behavior was entirely expected and represents the core advantage of semantic modelling over property graphs.

Query 1: List All Drugs

- **Explicit Pattern:** `?drug a ex:Drug`
- **Inferences Involved:** GraphDB utilizes `rdfs:subClassOf` and `rdfs:domain` reasoning.
- **Generated Triples:** Because `Antibiotic` and `AntiInflammatory` are subclasses of `Drug`, GraphDB automatically generates implicit triples such as `(ex:Ibuprofen rdf:type ex:Drug)` from the explicit fact `(ex:Ibuprofen rdf:type ex:AntiInflammatory)`. Furthermore, due to domain inheritance, instances acting as the subject of `treats`, `relieves`, or `worsens` generate implicit `rdf:type ex:Drug` triples.

Query 2: List All Diseases

- **Explicit Pattern:** `?disease a ex:Disease`
- **Inferences Involved:** GraphDB utilizes `rdfs:subClassOf` and `rdfs:range` reasoning.
- **Generated Triples:** Similar to Query 1, explicit instances of `InflammatoryDisease` and `InfectiousDisease` trigger the reasoner to generate implicit `(Instance rdf:type ex:Disease)` triples. Additionally, any instance acting as the object of an `affects` sub-property automatically generates a `rdf:type ex:Disease` triple due to the `rdfs:range` constraint.

Query 3: Retrieve All Drug-Disease Pairs

- **Explicit Pattern:** `?drug ex:affects ?disease`
- **Inferences Involved:** GraphDB primarily utilizes `rdfs:subPropertyOf` reasoning.
- **Generated Triples:** We did not explicitly write any facts using the `affects` property. However, because `treats`, `relieves`, and `worsens` are defined as sub-properties of `affects`, GraphDB evaluates explicit facts like `(ex:Amoxicillin ex:treats ex:BactInfection)` and generates the implicit triple `(ex:Amoxicillin ex:affects ex:BactInfection)`. This is precisely the expected behavior, successfully eliminating the need to hardcode specific interaction types into the SPARQL query.